

TeamFlow Systems Engineering Report

1. Executive Summary

Overview

TeamFlow is a web-based project management and collaboration platform designed for software engineering teams. The system integrates project planning, task management, Root Cause Analysis (RCA), notifications, reporting, and collaboration into a single platform. Instead of using multiple independent tools, TeamFlow provides a unified environment where project managers, developers, and reviewers can coordinate work efficiently.

The application follows a layered architecture consisting of a React frontend, an Express.js backend, and a PostgreSQL database. Communication between components occurs through REST APIs, while notifications are handled using an event-driven mechanism. The system supports both in-app and email notifications, ensuring users receive important updates without duplication.

The design emphasizes maintainability, scalability, and data consistency. Business rules such as reviewer approval for RCAs, task dependency validation, and notification deduplication ensure reliable project management.

Project Objectives

The primary objectives of TeamFlow are:

- Provide centralized project management.
- Improve collaboration among development teams.
- Support structured incident investigation using RCA.
- Maintain consistent project data using relational databases.
- Deliver notifications through both email and in-app channels.
- Produce dashboards and reports for project monitoring.

System Scope

The current implementation includes:

- User Authentication
- Project Management
- Task Management
- Project Members

- Root Cause Analysis
- Notifications
- Email Integration
- Reporting Dashboard
- CSV Export
- Theme Preferences

Features such as AI recommendations, WebSocket-based notifications, and Gantt charts are reserved for future versions.

2. Introduction

Software development teams often use different tools for project tracking, communication, issue investigation, and reporting. Maintaining data across multiple platforms introduces duplication, inconsistency, and reduced visibility into project progress.

TeamFlow addresses these challenges by integrating these capabilities into a single application. The platform enables teams to create projects, assign tasks, investigate incidents, notify team members, and monitor progress from a unified dashboard.

The system has been designed according to software engineering principles such as modularity, separation of concerns, layered architecture, and event-driven communication.

3. System Overview

The application consists of three major components.

Frontend

The frontend is developed using React. It provides the user interface for authentication, project management, task handling, dashboards, RCA creation, and notifications.

Responsibilities include:

- Displaying project information
- Collecting user input
- Calling backend APIs
- Rendering dashboards
- Showing notification indicators

Backend

The backend is developed using Node.js and Express.js.

Responsibilities include:

- Authentication
- Business rule validation
- Database operations
- Notification generation
- Email delivery
- CSV report generation

The backend follows a layered architecture.

Route

↓

Controller

↓

Service

↓

Model

↓

Database

This separation improves maintainability and allows each layer to focus on a single responsibility.

Database

PostgreSQL stores all application data.

Major tables include:

- Users

- Projects
- Tasks
- Project Members
- RCA
- Reviews
- Notifications
- Comments
- Attachments

The relational design maintains referential integrity using foreign keys and constraints.

4. Functional Features

Authentication

Authentication ensures that only authorized users can access the system.

Implemented features include:

- User Registration
- User Login
- JWT Authentication
- Password Hashing using bcrypt
- Protected Routes
- User Preference Management

Passwords are never stored in plain text. Instead, bcrypt hashing is used to protect user credentials.

Project Management

Projects act as containers for tasks, members, and investigations.

Users can:

- Create Projects
- Update Projects
- Delete Projects
- Invite Members
- Assign Roles

Each project automatically assigns the creator as the owner.

Task Management

Tasks represent individual units of work.

Each task supports:

- Title
- Description
- Status
- Priority
- Due Date
- Assignee
- Parent Task
- Attachments
- Comments

Task dependencies allow work sequencing and improve planning.

Root Cause Analysis (RCA)

The RCA module supports structured investigation of incidents.

Each RCA contains:

- Summary
- Timeline
- Severity
- Contributing Factors
- Corrective Actions
- Preventive Actions

The review process ensures investigations cannot be completed until all reviewers submit their decisions.

Notification System

Notifications keep users informed about important events.

Supported events include:

- Task Assignment
- Task Status Changes
- RCA Submission
- Review Decision

- Project Member Addition

Notifications are delivered through:

- In-app notification bell
- Email

Duplicate notifications are automatically suppressed using a deduplication key.

Reporting

The dashboard provides:

- Project Completion Rate
- Task Distribution
- Assignee Workload
- RCA Statistics
- Project Health

Reports can also be exported in CSV format.

5. Non-Functional Requirements

The system also satisfies several non-functional requirements.

Security

- JWT Authentication
- Password Hashing
- SQL Parameterized Queries
- Input Validation

Reliability

Foreign key constraints prevent inconsistent data.

Transactions ensure operations either complete entirely or rollback safely.

Maintainability

The layered architecture separates presentation, business logic, and data access.

Reusable React components reduce code duplication.

Scalability

The notification system uses an event-driven architecture, allowing additional handlers (SMS, push notifications, etc.) to be added with minimal changes.

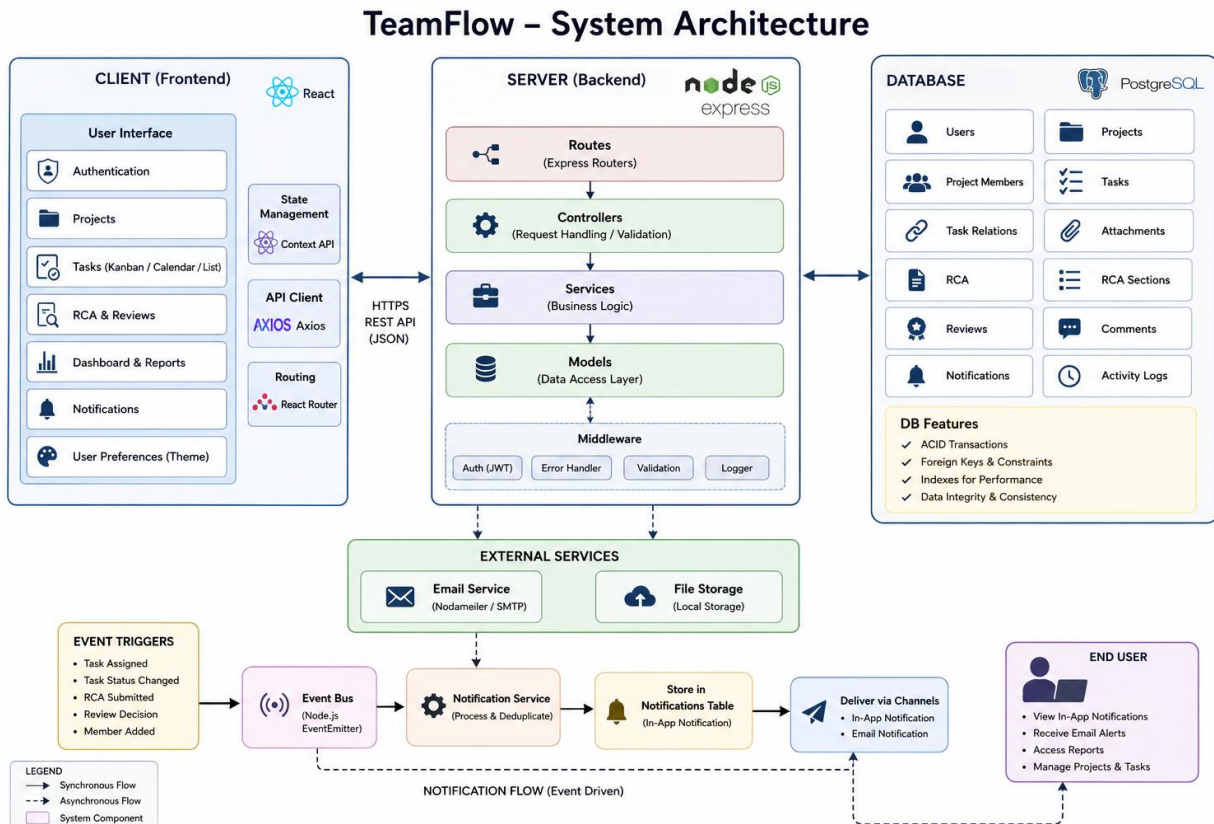
6. Architecture Summary

The system follows a **Layered Monolithic Architecture**.

Reasons for selecting this architecture include:

- Simpler deployment
- Easier debugging
- Lower operational complexity
- Suitable for medium-sized software engineering teams

The notification subsystem uses an **event-driven pattern**, reducing coupling between business logic and notification delivery.



7. Design Decisions Summary

Several important engineering decisions were made:

Decision	Reason
PostgreSQL	Strong relational consistency and foreign keys
React	Reusable component-based UI
Express.js	Lightweight REST API framework
JWT	Stateless authentication
Event-driven notifications	Loose coupling and extensibility
Layered architecture	Easier maintenance and testing

Each decision balances simplicity, maintainability, and future scalability.

8. Known Limitations

The current version has some intentional limitations:

- Notifications rely on polling instead of real-time updates.
- Email delivery does not include automatic retries.
- Attachments are stored locally.
- Theme preferences are browser-specific.
- Very large datasets may increase dashboard loading time.

These limitations were accepted to keep the initial implementation manageable.

9. Future Scope

Future versions of TeamFlow may include:

- Real-time notifications using WebSockets
- AI-based task prioritization
- Gantt charts and timeline planning
- Sprint management
- Workload prediction
- Cloud file storage
- Docker and Kubernetes deployment
- Cross-project analytics
- Mobile application support

These enhancements would improve scalability, collaboration, and user experience while building on the current architecture.

10. Conclusion

TeamFlow demonstrates the application of software engineering principles to the design of a collaborative project management system. The platform integrates project management, task execution, RCA workflows, notifications, and reporting into a single environment. By adopting a layered architecture, event-driven notifications, relational data modeling, and secure authentication, the system provides a maintainable and extensible foundation for managing software development activities.

The current implementation satisfies the core requirements of the assignment while leaving clear opportunities for future enhancements such as real-time collaboration, intelligent recommendations, and cloud-native deployment.